



ONE POEM SUFFICES · 一篇就够了

Agent Skills, 一篇就够了。

Composable Procedural Knowledge for Agents

Skills 本质是 Context, 取用方式是 Just-in-Time.

AUTHOR

@whisper

DATE

2026.06

READ

50 MIN

What is a Skill?

从格式标准上看，Skills 很简单：一个文件夹，一个 `SKILL.md`，再加可选的脚本和参考资料。它不像一个新的 Agent Framework，也不像 MCP 那样标准化外部能力的接入，更像一种可以被不同 Agent Harness 读取的轻量 packaging（参考 [Agent Skills Specification](#)）。

Skills 本质上就是 Context。技术上你完全可以在 Skills 文件夹里放任何类型的上下文：既可以是指导性上下文，也可以是信息性上下文，事实数据、API 文档、甚至 RAG 索引。三层结构对应的 token 量级：

- 第一层 **metadata** (`name` + `description`)：约 100 token，启动时预加载进 System Prompt
- 第二层 **SKILL.md 正文**：建议控制在 500 lines / 5,000 token 以内，被触发后才读
- 第三层 **bundled files** (`references/`、`scripts/`)：无上限，按路径按需打开

Tip

我与一些朋友交流时发现，很多人不知道 Skills 的 metadata 会常驻 System Prompt，就像不知道 MCP 也会把 Tools 的定义注入上下文。如果忽略概念的包装，把两者都理解成 Context，那它们出现在上下文窗口里就是自然的事。

一个最小的 Skill 长这样：

```
my-skill/SKILL.md
```

```
---
name: blog-review
description: Review blog drafts for style violations.
  Use when asked to "review my blog" or "check draft".
---
```

Blog Review

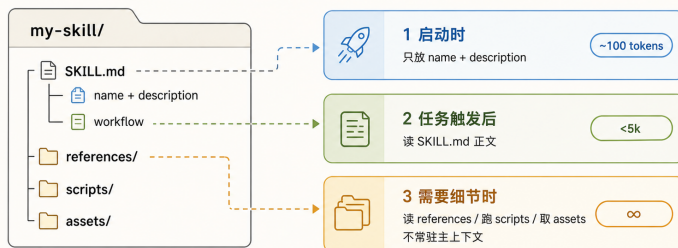
1. Read the draft and the style guide in references/.
2. Flag banned patterns; suggest rewrites in place.
3. Output a prioritized list of findings.

❓ scripts/ 是可执行的 tools，怎么能直接说 Skills 就是 Context?

工具的定义和用法本身也是 Context, MCP 就是它的标准化协议, scripts/ 可以看成它的本地版。Skill 从头到尾只负责提供 Context, “能跑”是 Harness/Sandbox 给的能力。

一个 Skill 文件夹，三层上下文

入口常驻，正文按需，大材料留在文件系统里。



💡 核心：小入口进上下文，大材料留在文件系统。

图 1 · Skill 的三层结构：metadata 常驻，body 与 references 按需拉取

Why do we need Skills?

Anthropic 对 Skill 的定义很简单：

“Skills 是一组组织好的文件，为 Agent 打包可组合的程序性知识。换句话说，它们就是文件夹。这份简单是刻意设计的。”*“Skills are organized collections of files that package composable procedural knowledge for agents. In other words, they’re folders. This simplicity is deliberate.”*

! Code is all we need

我们过去以为不同领域的 Agent 会长得很不一样，每个都需要自己的工具和脚手架。但底层的 Agent 其实比我们想象的更通用：代码不只是一个用例，而是通往数字世界的通用接口。

2.1 Skill 和已有机制的区别

机制	加载时机	典型场景
Prompt	当前对话内输入	一次性请求、临场修正
CLAUDE.md	进入项目即全量加载	项目结构、测试命令、代码风格
Skill	任务语义命中才加载	发布检查、数据库 migration

SkillsBench 的对照实验（84 个任务、11 个领域）给出了几项值得关注的结论：

发现	含义
人工精修的 Skill 平均提升 16.2%	Skill 有用，但前提是由人来把关质量
Healthcare 提升 ~52%，SWE 仅 4.5%	预训练覆盖越薄弱的领域，边际收益越大
模型自生成的 Skill 提升 0%	自己编写再自己执行，没有正收益

为什么叫 Procedural Knowledge?

它来自认知科学，跟 declarative knowledge (“知道什么”) 相对，指 “知道怎么做” 的那一类知识。不是每个 Skill 都是流程：一个只固定配色和字体的品牌设计 Skill，很难称为 Workflow Knowledge。

可跳过的深水区

接下来几段是基于 “非官方” 源码、对 Claude Code Memory 读写机制的拆解。如果你只关心差异结论，可以直接跳到下方，不影响后续阅读。就当是一次渐进式披露：这里是 [reference file](#)。

Warning

装第三方 Skill 约等于装可执行代码：来源不可信的 `scripts/` 存在供应链与注入风险，启用前先读一遍内容。

回到文章开始的两个观点：Skills 本质是 Context，Delivery 机制是 JIT / Pull。程序性知识从来不缺（文档里有、人脑里有、训练数据里也有），缺的是让它按需到位的机制与规范。更多讨论见[博客原文](#)。